

11-分布式

分布式面试题

1. 理论基础

问：CAP 理论是什么？

答： 分布式系统最多同时满足以下三个中的两个：

特性	说明
C (Consistency)	一致性，所有节点同一时刻看到相同数据
A (Availability)	可用性，每个请求都能得到响应
P (Partition tolerance)	分区容错，网络分区时系统仍能运行

实际：网络分区不可避免，所以只能在 C 和 A 之间权衡。 – CP：ZooKeeper、HBase – AP：Eureka、Cassandra

问：什么是分布式系统？和微服务有什么区别？

答： 分布式系统：多台计算机通过网络协同完成同一任务，对用户像一台超级计算机。

目的：高并发（水平扩展）、高可用（部分节点故障仍服务）。

vs 微服务：

对比	分布式	微服务
本质	物理部署方式	软件架构思想
解决	单机性能/可用性瓶颈	单体臃肿、难维护
关系	单体复制 10 份 = 分布式集群，≠ 微服务	微服务通常需分布式部署

引入代价：网络延迟、数据一致性、分布式事务/锁等复杂度。

问：BASE 理论是什么？

答： CAP 中 AP 的延伸： – BA (Basically Available)：基本可用，允许损失部分可用性 – S (Soft state)：软状态，允许中间状态 – E (Eventually consistent)：最终一致性

2. 分布式事务

问：分布式事务解决方案？

答：

方案	说明	缺点
2PC	准备 → 提交，强一致	同步阻塞、协调者单点
3PC	增加 CanCommit 超时机制	仍可能数据不一致
TCC	Try-Confirm-Cancel，业务层实现	开发复杂
Saga	长事务拆本地事务 + 补偿	无隔离性
本地消息表	业务 + 消息表同一事务	需定时扫描
事务消息	RocketMQ 半消息	依赖 MQ
Seata	AT/TCC/Saga/XA 模式	阿里开源

问：本地消息表方案是什么？

答：

1. 业务操作 + 插入消息表记录 → 同一本地 DB 事务
 2. 定时任务扫描 status=待发送 → 发送 MQ → 更新为已发送
 3. 消费者幂等消费
 4. 失败重试 + 人工对账

优点：不依赖 MQ 事务、实现简单。 缺点：需定时扫描、消息表膨胀、最终一致。

变体：RocketMQ 事务消息、Canal + MQ 删缓存。

问：Saga 模式是什么？

答：长事务拆为多个 本地事务，每步有 补偿操作（逆向撤销）。

类型	说明
编排式	中心协调器控制流程
编舞式	各服务监听事件自主推进

优点：无长锁、适合微服务长流程（下单→扣库存→支付→发货）。 缺点：无隔离性（脏读）、补偿逻辑复杂、需业务实现反向操作。

问：2PC 和 3PC 的区别？

答：

对比	2PC	3PC
阶段	准备 → 提交	CanCommit → PreCommit → DoCommit
阻塞	同步阻塞	引入超时，减少阻塞
单点	协调者宕机需等待	协调者宕机参与者可超时提交
数据一致	协调者宕机可能不一致	改善但仍不完美

问：TCC 是什么？

答：Try-Confirm-Cancel，业务层实现的三阶段：

阶段	说明
Try	预留资源（如冻结库存）
Confirm	确认执行（扣减库存）
Cancel	取消释放（解冻库存）

要求：幂等性、空回滚（Try 未执行收到 Cancel）、悬挂（Cancel 先于 Try）。

3. 分布式锁

问：分布式锁的实现方案？

答：

方案	优点	缺点
Redis（SET NX EX）	性能高	主从切换可能丢锁
Redisson	看门狗续期、红锁	依赖 Redis
ZooKeeper	强一致（临时顺序节点）	性能较低
数据库（for update）	简单	性能差、单点

问：Redisson 看门狗原理？

答：加锁成功后启动定时任务（默认 30s 过期，每 10s 续期到 30s）。业务完成手动 unlock 时取消续期。防止业务未完成锁就过期。

问：ZooKeeper 如何实现分布式锁？

答：利用 临时顺序节点 + Watcher：

1. 在 /locks 下创建 EPHMERAL_SEQUENTIAL 节点 (如 lock-000000003)
2. 判断自己是否序号最小 → 是则获得锁
3. 否 → watch 前一个节点, 等待其删除
4. 业务完成 delete 自己的节点 → 唤醒下一个
5. 客户端宕机 → 临时节点自动删除, 不会死锁

特点: 强一致 (ZAB 协议 CP), 性能低于 Redis。选型: 强一致选 ZK; 高并发选 Redis + Redisson。

问: 什么是 Fencing Token? 为什么 Redis 锁需要它?

答: Fencing Token: 单调递增的令牌, 客户端持锁期间每次操作携带当前 token; 存储层只接受 更大 token 的写请求。

场景: Redis 锁因主从切换/过期被误释放, 旧客户端仍以为持有锁继续写 → 数据错乱。

解决: 加锁时获取递增 token, 写 DB/资源时校验 token, 旧客户端 token 更小则被拒绝。

Redisson 可配合 RedissonLock + 业务层 token 校验; Martin Kleppmann 指出 Redlock 需 fencing 才安全。

4. 分布式 ID

问: 分布式 ID 生成方案?

答:

方案	优点	缺点
UUID	简单、无依赖	无序、36 字符太长
数据库自增	简单有序	单点瓶颈
Redis INCR	性能好	依赖 Redis
雪花算法	有序、高性能	时钟回拨问题
号段模式 (Leaf)	减少 DB 访问	需部署
UUID (百度 UidGenerator)	改进雪花	较复杂

问: 雪花算法结构?

答: 64 位:

0 | 41bit 时间戳 | 10bit 机器ID (5数据中心+5机器) | 12bit 序列号

- 每毫秒最多 4096 个 ID
- 时钟回拨: 拒绝生成或借用未来时间戳

5. 限流

问：常见限流算法？

答：

算法	原理	特点
固定窗口	单位时间内计数	简单，边界突刺
滑动窗口	细分时间窗口	平滑
漏桶	恒定速率流出	平滑输出
令牌桶	恒定速率放令牌	允许突发流量

实现：Guava RateLimiter（令牌桶）、Sentinel、Nginx limit_req。

6. 一致性

问：分布式一致性算法？

答：

算法	说明	应用
Paxos	经典共识算法，难理解	Chubby
Raft	易理解，Leader 选举 + 日志复制	Etcd、Consul
ZAB	ZooKeeper 原子广播	ZooKeeper

问：Paxos 和 ZAB 简要了解？

答：Paxos：经典共识算法，Proposer/Acceptor/Learner 角色，两阶段投票达成一致。理论完美但难实现。

ZAB（ZooKeeper Atomic Broadcast）： – 崩溃恢复：Leader 选举（类似 Raft） – 消息广播：Leader 广播提案，过半 Follower ACK 后提交 – 保证 顺序一致性 和 原子性

Raft vs Paxos：Raft 易理解（Leader 选举+日志复制），工程实现多（Etcd、Consul）。

问：什么是一致性 Hash？

答：将节点和数据映射到 Hash 环上，数据顺时针找到第一个节点。

优点：节点增减只影响相邻节点。

虚拟节点：解决数据倾斜，每个物理节点映射多个虚拟节点。

7. 服务治理

问：负载均衡策略？

答：– 轮询（Round Robin） – 加权轮询 – 随机 – 最少连接 – 一致性 Hash（会话保持） – 源地址 Hash

问：熔断、降级、限流的区别？

答：

概念	说明
限流	控制请求速率，超出的拒绝或排队
熔断	下游故障时快速失败，避免雪崩
降级	关闭非核心功能，保证核心可用

实现：Sentinel、Hystrix（已停更）、Resilience4j。

问：RPC 是什么？和 HTTP 有什么区别？

答：RPC（Remote Procedure Call）：像调用本地方法一样调用远程服务。

对比	RPC（Dubbo/gRPC）	HTTP（REST）
协议	自定义/TCP/HTTP2	HTTP/1.1
序列化	Protobuf/Hessian，紧凑	JSON，可读
性能	更高	较低
场景	服务内部调用	对外 API、浏览器

RPC 框架组件：注册中心、负载均衡、序列化、超时重试、熔断。

问：ZooKeeper 在分布式系统中做什么？

答：核心能力：分布式协调（CP 系统，基于 ZAB 协议）。

应用	原理
注册中心	临时节点存服务地址，宕机自动删除
分布式锁	临时顺序节点 + Watcher
配置中心	持久节点存配置，Watch 变更推送
Leader 选举	最小序号节点为 Leader（Kafka 旧版、HBase）
命名服务	树形目录统一管理

Watch 特点：一次性触发，需重新注册。

注意：ZK 不适合海量读写，适合低频协调场景；注册/配置现多被 Nacos、Etcd 替代。

8. Raft 共识算法

问：Raft 选举流程详解？

答：

角色：Leader（处理请求）、Follower（被动接收）、Candidate（选举中）

选举触发：Follower 超时未收到 Leader 心跳 → 转 Candidate

选举过程：

1. 任期 Term +1, 投票给自己
2. 并行向其他节点发送 RequestVote RPC（携带 Term、lastLogIndex、lastLogTerm）
3. 多数派 ($>N/2$) 同意 → 成为 Leader
4. Leader 定期发心跳（AppendEntries RPC）维持权威

投票规则：

- 每个任期最多投一票
- 候选者日志至少和自己一样新（lastLogTerm 更大, 或 Term 相同 index 更大）

脑裂防护：必须获得多数派选票，2 节点集群无法容忍 1 节点故障（需至少 3 节点）。

任期单调递增：收到更高 Term 的 RPC 立即转 Follower。

问：Raft 日志复制流程？

答：

1. Client 请求 → Leader
2. Leader 追加日志条目（未提交状态）
3. 并行 AppendEntries 到 Follower
4. 多数派确认写入 → Leader 提交（commitIndex 推进）
5. Leader 通知 Follower 提交
6. 应用到状态机，返回 Client

日志一致性：Follower 日志与 Leader 冲突时，从后向前找匹配点，截断冲突后续。

9. Seata 分布式事务

问：Seata AT 模式原理？

答：AT（Automatic Transaction），对业务无侵入，基于数据源代理和undo log。

一阶段：

1. 解析 SQL，查询前镜像 (before image)
2. 执行业务 SQL
3. 查询后镜像 (after image)
4. 生成 undo log，提交本地事务（释放行锁）

二阶段-提交：

异步删除 undo log

二阶段-回滚：

根据 undo log 生成反向 SQL 恢复数据

隔离性：全局锁（Global Lock）防止脏写——本地事务提交前需获取全局锁，其他全局事务修改同一行需等待。

优点：无侵入、性能好。 **缺点：**依赖数据库本地事务，不支持 NoSQL。

问：Seata 其他模式对比？

答：

模式	原理	侵入性	场景
AT	自动 undo log	低	关系型 DB
TCC	Try-Confirm-Cancel	高	需精细控制资源
Saga	正向服务 + 补偿	中	长事务
XA	数据库 XA 协议	低	强一致，性能差

10. 分布式 Session

问：分布式 Session 有哪些方案？

答：

方案	原理	优缺点
Session 复制	Tomcat 集群同步 Session	简单，节点多时复制开销大
Session 粘滞	Nginx ip_hash / 一致性 Hash	节点宕机 Session 丢失
Session 集中存储	Redis/Memcached 存 Session	主流 ，需 Session ID 传递
JWT 无状态	Token 自包含用户信息	无法即时失效，适合 API
Spring Session	抽象层，底层接 Redis	对应用透明

推荐：Redis + Spring Session + Cookie 传 Session ID。

11. 接口幂等

问：如何实现接口幂等？

答：幂等：同一请求执行多次，结果与执行一次相同。

方案	实现
唯一索引	数据库 <code>UNIQUE(order_no)</code> 防重复插入
Token 机制	先获取 Token 存 Redis，提交时校验并删除
状态机	订单状态 待支付→已支付，已支付则直接返回
分布式锁	<code>SETNX requestId</code> 保证只处理一次
乐观锁	<code>UPDATE ... WHERE version = ?</code>
去重表	记录 (biz_type, biz_id) 唯一

HTTP 幂等：GET/PUT/DELETE 天然幂等，POST 需业务保证。

MQ 消费幂等：消息 ID + Redis/DB 去重。

12. 服务治理深入

问：服务降级、熔断、限流的区别和关系？

答：

概念	触发条件	目的	比喻
限流	请求量超阈值	保护系统不被压垮	景区限流
熔断	下游错误率/慢调用超阈值	快速失败，防止雪崩	保险丝
降级	系统压力大或下游不可用	关闭非核心，保核心	春运停售非必要车次

关系：限流是入口防护 → 熔断是下游故障隔离 → 降级是兜底策略。常组合使用（Sentinel 三者都支持）。

降级策略：返回默认值、读缓存、返回静态页面、排队异步处理。

问：限流在分布式环境下如何实现？

答：

方案	说明
网关层	Nginx limit_req、Kong、APISIX
应用层	Sentinel、Guava RateLimiter
分布式限流	Redis + Lua 脚本（滑动窗口/令牌桶）
服务网格	Istio Envoy 限流

Redis 滑动窗口 Lua：

```
-- 按时间窗口 ZSET 记录请求时间戳，过期清理，计数与阈值比较
```

13. 分布式 ID 深入

问：雪花算法时钟回拨如何处理？

答：

```
64bit = 1符号 + 41时间戳(ms) + 10机器ID + 12序列号
```

时钟回拨：系统时间被 NTP 同步倒退，可能生成重复 ID。

处理策略：

策略	说明
拒绝生成	回拨时间短 (<5ms) 则等待追上
借用未来	回拨时序列号继续递增，时间追上后等待
扩展位	回拨期间用扩展 bit 区分
人工介入	大幅回拨告警，停机处理

美团 Leaf：号段模式 + 雪花双方案；百度 UidGenerator：RingBuffer 缓存。

问：号段模式（Leaf Segment）原理？

答：

```
1. 数据库表存 MAX_ID，每次取一个号段（如 1000 个）
2. 内存中 INCR 分配，用完再取下一个号段
3. 双 Buffer：一个用完前异步加载下一个，避免 DB 瓶颈
```

优点：ID 趋势递增、DB 压力小。 缺点：依赖 DB，号段浪费（重启）。

图解加深

专题	要点
CAP/BASE	分区容错下的 C/A 权衡
Raft	选举、日志复制、安全性证明
一致性 Hash	虚拟节点、数据倾斜
分布式锁	Redis Redlock 争议、ZK 临时顺序节点
Seata AT	一阶段提交、undo log、全局锁
雪花算法	bit 分配、时钟回拨
ZK 应用	注册/锁/配置/选主
Fencing Token	防锁过期后旧客户端写

大厂追问

1. **2PC 和 Seata AT 有什么区别？** 2PC 二阶段才释放锁，阻塞长；AT 一阶段提交释放本地锁，靠全局锁防脏写，二阶段异步。
2. **Redis 分布式锁 Redlock 安全吗？** Martin Kleppmann 质疑：主从切换可能丢锁；Redis 官方回应需正确实现+fencing token。
3. **接口幂等和防重有什么区别？** 幂等强调多次执行结果相同；防重强调拒绝重复请求，是幂等的一种实现手段。
4. **如何实现跨服务的登录态共享？** SSO (CAS/OAuth2)、Gateway 统一鉴权、JWT 或 Redis Session。
5. **Eureka 和 Nacos 注册发现对比？** Eureka AP 优先，自我保护；Nacos 支持 AP/CP 切换，配置中心一体。
6. **分布式事务一定要强一致吗？** 大多数业务最终一致即可 (BASE)，强一致用 2PC/XA 性能差，优先 Saga/事务消息/TCC。
7. **Gossip 协议是什么？** 节点随机选 peer 交换状态，最终传播到全网；适合成员发现、故障检测 (Cassandra、Redis Cluster)。
8. **注册中心 Eureka 和 Nacos 区别？** Eureka AP 优先、自我保护不剔除不健康实例；Nacos 支持 AP/CP 切换，集成配置中心，Spring Cloud 主流已转 Nacos。
9. **API 网关和注册中心有什么区别？** 注册中心是服务间发现 (谁在哪)；API 网关是对外统一入口 (路由、鉴权、限流、聚合)，如 Spring Cloud Gateway、Kong。